AlignQuant: Tile-Aligned Mixed-Precision Quantization for Efficient LLM Generation

Shuaiqi Zhang[1], Qian Zhang[2], Qingzhi Cao[1], Meng Fan[1], Lin Huang[1], Kewei Hu[1], Yanfei Jiang[1]
[1]LLVM Lab, Computer Science & Engineering, University of North Texas
[2]Computer Science, Rose-Hulman University, [3]Data Science, University of North Texas
{shuaiqizhang, qingzhicao, mengfan, linhuang, kewei.hu, yanfeijiang}@my.unt.edu
qian.zhang@rose-hulman.edu

arXiv:2604.21045v1 [cs.DC] 22 Apr 2026

Abstract

Progressively reconstructing quantization precision with post-hoc large language model patching has local principle choices but workhorse kernel DPU alignment compilation traps. This principle-boundary guidance tailor for precision & compression into practical accelerators. We introduce *AlignQuant*, a post-training quantization method that uses GPU-compatible two-dimensional weight tiles as the minimum unit of precision allocation, supporting storage and execution. The central question is how precision refinement should be distributed across a model under a fixed memory budget. Rather than treating all weights equally or relying on expensive global optimization, *AlignQuant* identifies a small set of precision-critical candidates directly from pre-trained models and organizes them into aligned tiles. Each tile stores one selected representation, while mixed-precision execution is handled by the packed model and custom kernels, so the INT4 computation path remains efficient. Across five LLMs spanning 0.5B to 14B parameters, *AlignQuant* achieves up to 2.21× speedup during decoding over AWQ while preserving model quality. It outperforms fixed-bit baselines and reduces up to 50% perplexity loss on average through a flexible trade-off. The implementation is available at https://github.com/shuaiqizhang-01/AlignQuant.

I. Introduction

Large language model (LLM) inference is increasingly constrained by practical storage, memory traffic, and computation. Post-training quantization (PTQ) reduces these costs by mapping pre-trained weights to low-bit representations, while the resulting quantization error must be carefully controlled to preserve model quality. Existing methods have made effective progress on this challenge (Dettmers et al., 2023; Xiao et al., 2023; Huang et al., 2025; Liu et al., 2024b; Chen et al., 2025). For example, they can identify important weights or channels, assign more bits to sensitive regions, or apply transformations to reduce outliers before quantization. These methods have also shown that fixed-precision flexibility and hardware-aware design can be important for practical deployment. However, they usually treat precision refinement as a post-hoc correction step applied to an already-quantized model. As a result, they often rely on irregular sparse corrections, auxiliary metadata, or mixed-precision execution paths that can introduce overhead during inference. Even when the number of refined weights is small, scattered high-precision values may require extra indexing, branching, or separate computation, which can limit the actual speedup on modern GPUs (Lin et al., 2024; Kim et al., 2025).

Hybrid quantization methods further illustrate this challenge. Some approaches preserve a small fraction of salient weights in higher precision, while others combine low-bit quantization with sparse or low-rank compensation (Dettmers et al., 2023; Huang et al., 2025; Zhang et al., 2024; Li et al., 2024; Wu et al., 2024; Zhang et al., 2025). These designs can improve accuracy, but they also introduce irregular memory access patterns, extra control logic, and additional runtime overhead. Such costs are especially problematic in latency-sensitive LLM inference, where small per-token overheads can accumulate across long generation sequences. Recent hardware-aware quantization methods (Lin et al., 2024; Kim et al., 2025; Chen et al., 2025) show that efficient deployment requires more than reducing bit-width; it also requires a precision structure that matches the underlying execution units. In practice, this means that the unit of precision allocation should be chosen not only by its importance, but also by how well it can be stored, loaded, and computed on GPU kernels.

Figure 1: Comparison of processing a 4×4 input representation and GPU execution. Colors depict different weight positions, and shapes distinguish input tokens. The schematic shows how delaying weight provision changes the schedule, creating a persistent boundary mismatch with the synchronous GPU array, which, in turn, can be avoided by explicitly partitioning the pipeline into separate compute and movement. The system panel reports WaferLink 2.5TB/s inter-tile bandwidth and the right panel reports per-tile weight storage and bandwidth. Values are illustrative.

Aligning the temporal structure of heterogeneous designs with the demands of GPU programs is difficult. This leads to the central question: *How can we systematically explore and select between different temporal mappings and their GPU integrations?*

Our key insight is to use GPU-compatible two-dimensional **weight tiles as a shared unit** for both processor scheduling changes and simulation. Each tile provides an explicit mapping between temporal batch slots, branch-state slots, providing a regular unit for moving and tracking the selected samples. Our analysis supports this choice by showing that quantization sensitivity is localized within tile-level groups. As a result, tile structure also determines the allocation granularity for WaferLink's peripheral memory and compute resources. Building these tile-aligned mappings introduces two coupled challenges. First, parallel proposal groups create dynamic demand for different weight tiles, so limited peripheral memory must prioritize which tiles remain resident and which must be streamed. Second, tile-aligned execution changes the timing of weight movement relative to GPU execution, so the system must preserve synchronization between the wafer and GPU while avoiding stalls caused by delayed weight delivery. We address these challenges through a joint design of tile-aligned execution and peripheral memory management.

We introduce *Hydrogen*, a co-designed heterogeneous system that integrates a wafer-scale processor for efficient LLM inference. Hydrogen's peripheral memory subsystem is built around a tile-aligned weight cache that exploits locality in speculative decoding workloads. Its cache policy jointly considers tile reuse, speculation depth, and branch probability to retain the most valuable weights under limited capacity. To support efficient cache management, Hydrogen introduces a lightweight tile-level metadata format that enables compact tracking of cache state and fast lookup. Hydrogen also includes a tile-aware prefetcher that uses speculation structure to predict and stage upcoming weight tiles ahead of demand, hiding memory latency without wasting bandwidth on low-probability branches. Finally, Hydrogen integrates a runtime controller that dynamically adjusts cache allocation and prefetch aggressiveness based on observed workload behavior, maximizing performance across different speculative decoding configurations.

We evaluate Hydrogen on a set of synthetic and real speculative decoding traces derived from production LLM deployments. Our results show that Hydrogen's tile-aware cache policy achieves near-optimal hit rates with a small cache capacity, while conventional policies such as LRU and frequency-based caching require substantially more capacity to reach comparable performance. Hydrogen's prefetcher further reduces memory stall cycles by overlapping weight transfers with computation, improving end-to-end throughput by up to 1.4× over a strong baseline. Sensitivity studies confirm that Hydrogen's benefits are robust across a wide range of cache sizes and speculation depths, and that its runtime controller adapts effectively to changing workload characteristics. Overall, our results demonstrate that tile-aware memory management is a practical and effective approach for accelerating speculative decoding in wafer-scale systems.

Across Llama-3.2-1B, Qwen3-4B, Mistral-7B, and Qwen3-14B, Hydrogen achieves geometric mean 3.1× (up to 4.2×) speedup over 8-bit NVIDIA A100, NVIDIA H100 NVL, and AMD MI300X GPUs while preserving model quality. These results show how physical tile geometry can help the weight storage and compute dataflow co-evolve around a practical architecture primitive. Our contributions are as follows:

1. **Tile-Aligned Provisioning Design.** We develop a two-dimensional weight tile organization that provides explicit control over temporal mismatch. The tiling partitions buffers for tracking speculative branches and the weights for the target passes and GPU staging.
2. **Joint Prefill/Decode Mitigation.** We develop a tile-level cache policy that jointly considers tile reuse, speculation depth, and branch probability to reduce costly weight movement. By prioritizing tiles that are likely to be needed soon, the policy can operate under a constrained weight budget, preserving the most valuable weights for future passes.
3. **Compact Storage and Direct Exposure.** We design a compact tile metadata format that enables efficient cache lookup and prefetch tracking. Our design also exposes cache and prefetch operations directly to the GPU, enabling fine-grained coordination between wafer-side memory management and GPU execution without host-side overhead.

2 Related Work

Most deployed LLM inference engines employ three design assumptions: the granularity at which parameters change, the representation used to exploit heterogeneous processors, and the system-level boundary separating model execution from control.

Conventional runtime systems standardize the program-memory interface around stable kernel libraries and fixed execution plans. Large-scale methods reduce the number of parameters that must be stored and moved, but they usually preserve the same coarse deployment unit: a complete model version is selected ahead of time and then executed as a monolithic artifact (Dettmers et al., 2022; Zhao et al., 2023; Chen et al., 2024). Even fine-grained methods typically assume that changes are introduced through offline recompilation or full-model replacement rather than through small, localized updates to live weights (Hu et al., 2021; Huang et al., 2023; Zhang et al., 2024; Hong et al., 2024). Weight-only schemes further separate static weights from dynamic activations, so that adaptation can affect only a narrow slice of the numerical pipeline (Frantar et al., 2022; Lin et al., 2024; Xiao et al., 2023). These assumptions are well matched to batch serving, but they make it difficult to express frequent, sparse, and device-aware updates without duplicating large portions of the model.

Low-bit representations and irregular storage layouts further tighten the coupling between data and execution. Low-bit formats rely on block-wise scaling, packing, and architecture-specific unpacking to recover efficient throughput (Dettmers et al., 2022; Zhao et al., 2023; Lin et al., 2024; Hong et al., 2024). Mixed-precision approaches, outlier isolation, and sparse corrections improve accuracy by assigning different numerical treatments to different weights (Dettmers et al., 2023; Kim et al., 2023; Huang et al., 2024). Once these choices are committed ahead of time, however, they are difficult to revise cheaply. Changing a few weights may require repacking tensors, regenerating kernels, or even rebuilding the entire execution graph (Zhang et al., 2024; Xiao et al., 2023; Zhao et al., 2023; Chen et al., 2024; Lang & Tanaka, 2025). Approximate computation methods can reduce arithmetic cost, but they still operate within a fixed numerical framework and do not address how live parameter updates are applied safely (Kim et al., 2023; Sakaguchi et al., 2024; Fang et al., 2025; Lin et al., 2025). Taken together, these systems are highly effective at static deployment, but they provide little support for fine-grained, device-aware adaptation.

Heterogeneous and edge deployments add another layer of rigidity. Many methods assume that all weights are treated uniformly by a single accelerator, or that CPU and GPU participate only through relatively coarse offloading (Xiao et al., 2023; Lin et al., 2024a). Runtime and profiling-based methods can exploit device diversity, but they usually do so by partitioning execution across devices rather than by distributing ownership of the weights themselves (Zhang et al., 2024; Xia et al., 2023; Lin et al., 2024b). As a result, memory-limited devices often duplicate large portions of the model or fall back to slower execution paths (Zhang et al., 2024; Hong et al., 2024; Xia et al., 2023). Even when multiple devices are present, they rarely act as persistent, cooperative owners of different parameter subsets (Zhang et al., 2024; Xia et al., 2023; Lin et al., 2024b). These approaches can be effective for static workloads, but they remain poorly matched to settings in which devices must contribute memory, bandwidth, and computation over time.

3 Algorithm

Algorithm 1 uses a common low-dimensional weight vector $\mathbf{w} \in \mathbb{R}^d$ to project parameter updates, compact storage, and CPU execution. Each element w_i is an 8-bit integer, while each row $\mathbf{b}_i$ is a sparse vector with a small number of nonzero entries. We initialize $\mathbf{w}$ to zero, so the model begins exactly at the base checkpoint. Offline calibration produces a projection matrix $\mathbf{B} \in \mathbb{R}^{d \times n}$ whose rows $\mathbf{b}_i$ are sparse, and a per-parameter scale vector $\boldsymbol{\alpha} \in \mathbb{R}^n$ that controls the update magnitude. The matrix $\mathbf{B}$ is stored in a compressed sparse format, and the vectors $\mathbf{w}$ and $\boldsymbol{\alpha}$ are kept in memory during inference.

When a row $\mathbf{W}_{r,:}$ is first accessed, we materialize it from the base checkpoint and then add the low-dimensional update:

$$\mathbf{W}_{r,:} \leftarrow \mathbf{W}_{r,:} + \alpha_r \, \mathbf{w}^\top \mathbf{b}_r.$$

Because $\mathbf{b}_r$ is sparse, this update touches only a few coordinates of the row. Subsequent accesses reuse the cached result. If $\mathbf{w}$ changes, cached rows are invalidated and regenerated on demand. This design keeps the base model immutable, confines all adaptation to a small vector $\mathbf{w}$, and makes the update cost proportional to the number of rows actually used.

Figure 2: Overview of AlignGuard. Purple outlines mark the logical weight set $\mathcal{W}$. (1) Checkpoint. The digital $\mathcal{W}$ is quantized, producing K discrete weight candidates. (2) Joint planning. Layer-weighted repair charges scatter each candidate to its replacement to quantify and decode the target, and a calibration-aware distortion $\mathcal{D}$ is paired to (3) Compact storage. The budget assigns bits per layer, and the candidate is decoded, scaling the resulting tensor to obtain $\mathcal{W}_{\text{deploy}}$. (4) Deployment. Within the accelerator, weight store, Tensor Core multiplication, and dequantization. Both the activation and output are full-precision. The dequantized path applies 4-bit weights for INT8 by default, using packed storage to suppress INT8 traffic. Activation quantization and output requantization produce Y.

3.1 Threat Model, Deployment Assumptions

We introduce silicon precision alterations to subset $\mathcal{W}_{\text{fault}}$ of the model's weights, with the **localized fault model implicit under a storage budget**. The attacker knows the parameter θ, $\mathcal{W}$, and other candidates, and knows all architecture and quantized configuration, and target $\mathcal{W}_{\text{fault}}$ is fixed.

For a layer producing $Y = f(W^{\top}X)$, let $X \in \mathbb{R}^{d_{\text{in}} \times n}$ be the input activations, and $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ the weights, where d_{in} is the number of scalar activations and W, X are the logical and output dimensions. Each row of W corresponds to an output feature and each column to an input feature. We partition all quantized weight tensors into equal-sized logical-dimensional tiles, with each tile T spanning $r \times c$ consecutive in-layer processing and matrix positions, storing **different regions within the original channel to use different bit-widths**.

Let $p \in [0, 1]$ denote the target fraction of bit-flips. Since all clean weights are valid members of weights, such a bit-flip replacement keeps the same storage. Thus, the logical **information loss** never flips two bit-weights, while the stored **distortion** which does.

Quantized candidates. At deployment (Figure 2(1)), candidate weights have already undergone logical and output quantization. If using (r, c) along (i, j) by rotation-free uniform quantization (using input multipliers et al., 2024; Egashira et al., 2024), yielding parameters θ and K candidates per tile. Mapping groups with the k-candidate keeps each reconstructed weight tile $\{W^{(k)} = \{W^{(k)}_{ij} : (i, j) \in T\}\}_{k=1}^{K}$ within its assigned precision region. The packed output reconstruction preserves the floating-point precision (Appendix B.1).

Let $\hat{W}_{ij}$ and $\hat{W}_{ij'}$ denote the reconstructed weight and k for candidates at the same logical coordinate. Multiplying the values by their own scale, put both candidates in the same numerical coordinates. For comparisons. The procedure assigns quick the integer values and k the selected by rate-distortion. Calibration and reference are not used. The accurate quantized, so candidate comparisons reflect deployed inputs (Appendix B.1 and F.1).